EVALUACIÓN PARCIAL N° 5 · DSY1107 DESARROLLO CLOUD NATIVE I

🚗 CARHUB

Documentación Técnica y Arquitectura de Streaming con Apache Kafka

Coexistencia Resiliente con Clúster RabbitMQ, Spring Boot 3.5 y Docker

Asignatura:	Desarrollo Cloud Native I (DSY1107)
Institución:	Duoc UC · Escuela de Informática y Telecomunicaciones
Docente:	Jorge Niochet
Integrantes:	Bruno Soto · Bayron Fuentes · Cristopher Candia · Matias Reyes
Puerto Kafka Asignado:	**5030** (Broker 1) / **5031** (Broker 2)
Fecha:	Noviembre 2026

1. Resumen Ejecutivo y Alcance

El presente informe documenta en detalle la integración de sistemas de **Streaming y Analítica en Tiempo Real con Apache Kafka** dentro del ecosistema de microservicios **CarHub**. Esta fase evolutiva cumple con la totalidad de los criterios establecidos en la pauta de evaluación parcial, complementando la infraestructura previamente construida sobre RabbitMQ, MySQL 8.0 y Microsoft Entra ID (Azure AD).

El diseño garantiza la **convivencia no destructiva** entre ambos paradigmas de mensajería: RabbitMQ como carril transaccional crítico para tickets y soporte con Dead Letter Queues (DLQ), y Apache Kafka como plataforma distribuida de streaming para telemetría vehicular y analítica de compras con alta disponibilidad y persistencia configurable.

2. Justificación Arquitectónica: RabbitMQ vs. Apache Kafka

Dimensión	RabbitMQ (Mensajería Transaccional) 🐇	Apache Kafka (Streaming & Analítica) ⚡
Patrón de Diseño	Colas de trabajo (Smart Broker / Dumb Consumer). Enrutamiento dinámico Direct y Topic.	Registro distribuido inmutable basado en logs append-only (Dumb Broker / Smart Consumer).
Responsabilidad CarHub	Emisión puntual de comprobantes/facturas y tickets de soporte técnico postventa.	Streaming analítico de compras en tiempo real y telemetría de interacción con vehículos.
Ciclo de Vida	El mensaje es consumido y destruido tras la confirmación manual (`basicAck`).	Los eventos persisten en disco según retención (días), soportando replay y múltiples consumidores.
Tolerancia a Fallos	Confirmación manual (`basicNack` con `requeue=false`) desviando a DLX y DLQ.	`DefaultErrorHandler` con reintentos programados (`FixedBackOff`) y Dead Letter Topic (DLT).
Escalabilidad	Equitativa entre consumidores activos de una cola mediante Round-Robin.	Particionamiento estricto por clave (`partitionKey`) y balanceo por `Consumer Groups` .

> **Conclusión de Diseño:** Ninguna tecnología reemplaza a la otra; ambas se potencian. Por ejemplo, al concretarse una adquisición vehicular en `backend-compra` , el microservicio emite un ticket transaccional a RabbitMQ para asegurar su procesamiento unitario y, en paralelo, emite un evento analítico a Kafka hacia `carhub.ventas.streaming` para alimentar tableros de control en vivo.

3. Topología de Infraestructura Docker y Puertos

Toda la infraestructura se despliega en contenedores interconectados mediante la red puente `carhub-infra-net` en `docker-compose.yml`.

Servicio / Contenedor	Puerto Host	Puerto Interno	Rol e Infraestructura
`zookeeper-carhub`	2181	2181	Coordinador del clúster Kafka, elección de líderes y metadatos.
`kafka1-carhub`	**5030**	29092	**Broker 1 (Líder Clúster)**. Puerto 5030 configurado como listener externo.
`kafka2-carhub`	**5031**	29093	**Broker 2 (Réplicas)**. Provee el factor de replicación 2 exigido por pauta.
`kafka-ui-carhub`	**8085**	8080	Panel visual interactivo para explorar tópicos, particiones, réplicas y grupos.
`rabbitmq1` / `rabbitmq2`	5672 / 5673	5672	Clúster RabbitMQ sincronizado de 2 nodos con paneles en 15672 y 15673.
`mysql-carhub`	3306 / 3307	3306	MySQL 8.0 con 6 bases de datos independientes.

> **Configuración del Puerto 5030:** Se configuró la variable `KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka1:29092,PLAINTEXT_HOST://localhost:5030`, permitiendo a cualquier cliente externo o herramienta local interactuar directamente con Kafka a través de `localhost:5030`, mientras los microservicios dockerizados dialogan internamente a alta velocidad.

4. Matriz de Tópicos, Particionado y Políticas de Retención

Nombre del Tópico	Particiones	Factor Réplica	Retención (`retention.ms`)	Cleanup	Productor	Consumer Group
`carhub.ventas.streaming`	3	2	7 Días (604.800.000 ms)	`delete`	`backend-compra` (8065)	`carhub-analytics-ventas-group`
`carhub.actividad.vehiculos`	3	2	3 Días (259.200.000 ms)	`delete`	`backend-carhub-vehiculo` (8062)	`carhub-telemetria-vehiculos-group`

Justificación de Parámetros:

- **3 Particiones por tópico:** Permiten balanceo de carga real sobre hilos de consumo paralelos (`concurrency = 3`), distribuyendo los mensajes según la llave del evento.
- **Factor de Replicación 2:** Respaldado en los 2 brokers activos, garantizando tolerancia ante la eventual caída de un broker.
- **Retención diferenciada:** Las transacciones de venta poseen mayor valor histórico (7 días para reportería semanal) mientras que los eventos de telemetría y navegación vehicular tienen una vida útil más corta (3 días).

5. Detalle de Componentes Implementados por Microservicio

A. Microservicio Administrador y Analítico: `backend-carhub` (Puerto 8066)

- **`KafkaConfig.java`:** Configuración central de Spring Kafka. Declara los beans `NewTopic` con retención y factor de replicación, inicializa el `AdminClient` oficial y define el `ConcurrentKafkaListenerContainerFactory` con concurrencia de 3 hilos.
- **Mecanismo de Resiliencia:** Incorpora `ErrorHandlingDeserializer` envolviendo a `JsonDeserializer` para aislar mensajes defectuosos o payloads corruptos sin bloquear el hilo. Implementa `DefaultErrorHandler` con `FixedBackOff(1000L, 3L)` que ejecuta hasta 3 reintentos antes de desviar y loguear contextual mente el error.
- **`KafkaAdminService.java` & `KafkaAdminController.java`:** Expone una API REST completa para gestión del clúster:
  - `POST /api/v1/admin/kafka/topics`: Creación dinámica con particiones, réplicas y retención.
  - `GET /api/v1/admin/kafka/topics`: Listado detallado de tópicos.
  - `GET /api/v1/admin/kafka/topics/{name}`: Detalle de particiones, líderes e ISR.
  - `PATCH /api/v1/admin/kafka/topics/{name}`: Alteración dinámica de particiones y configs.
  - `DELETE /api/v1/admin/kafka/topics/{name}`: Eliminación segura de tópicos.
  - `GET /api/v1/admin/kafka/consumer-groups`: Inspección de grupos, estados (`STABLE`) y miembros.
  - `GET /api/v1/admin/kafka/analytics/...`: Consulta de métricas agregadas en memoria.
- **Consumidores con Separación de Dominio:** `VentasKafkaConsumer.java` y `VehiculoActividadKafkaConsumer.java` reciben el registro vía `@KafkaListener` y delegan inmediatamente la lógica de negocio a `AnaliticaVentasService` y `AnaliticaVehiculosService`, evitando acoplar la infraestructura de mensajería con las reglas de dominio.

B. Microservicio Productor de Ventas: `backend-compra` (Puerto 8065)

- **`KafkaProducerConfig.java`:** Instancia el `KafkaTemplate<String, Object>` configurando `acks=all`, `retries=3` e idempotencia habilitada.
- **`VentaStreamEventDto.java`:** DTO estructurado con `idVenta`, `idUsuario`, `marca`, `modelo`, `montoTotal`, `metodoPago`, `canal` y `fechaEmision`.
- **`VentaKafkaProducer.java`:** Despacha asíncronamente con callbacks (`CompletableFuture`), particionando los eventos por `idUsuario` o factura para garantizar orden secuencial de compras por cliente.
- **Integración en `compraService.java`:** Tras persistir la factura en MySQL, ejecuta en secuencia la publicación en RabbitMQ (ticket de compra) y en Kafka (streaming analítico).

C. Microservicio Productor de Telemetría: `backend-carhub-vehiculo` (Puerto 8062)

- **`VehiculoActividadEventDto.java`:** DTO para eventos de actividad vehicular (acciones: `VISTO`, `CREADO`, `ACTUALIZADO`).
- **`VehiculoActividadKafkaProducer.java`:** Emite eventos particionados por `idVehiculo` hacia `carhub.actividad.vehiculos`.

- **Integración en `VehiculoService.java`:** Conecta el flujo en los métodos `obtenerVehiculoPorId` (vistas), `guardarVehiculoModel` (creaciones) y `actualizarVehiculo` (modificaciones).

6. Herramienta CLI de Infraestructura (`dockers/kafka-admin.sh`)

Para facilitar la verificación y responder al criterio de scripts de infraestructura de la pauta, se incorporó el script `kafka-admin.sh` con comandos ejecutables directamente en la terminal:

```
# 1. Otorgar permisos de ejecución
chmod +x dockers/kafka-admin.sh

# 2. Listar todos los tópicos del clúster
./dockers/kafka-admin.sh listar-topicos

# 3. Describir particiones, réplicas y políticas de retención
./dockers/kafka-admin.sh describir-topico carhub.ventas.streaming

# 4. Crear un tópico de prueba con 3 particiones y réplica 2
./dockers/kafka-admin.sh crear-topico carhub.telemetria.talleres 3 2 86400000

# 5. Listar e inspeccionar Consumer Groups y Lag
./dockers/kafka-admin.sh listar-grupos
./dockers/kafka-admin.sh describir-grupo carhub-analytics-ventas-group

# 6. Escuchar eventos en vivo desde el offset inicial
./dockers/kafka-admin.sh consumir carhub.ventas.streaming

# 7. Diagnosticar estado de brokers y Zookeeper
./dockers/kafka-admin.sh estado-cluster
```

7. Matriz de Cumplimiento de la Pauta de Evaluación N° 5

Indicador de Evaluación	Pond.	Nivel Alcanzado	Evidencia en el Proyecto
Configuración completa cliente Kafka (Bootstrap servers, serializers, properties).	12%	**100% (Muy Bueno)**	`KafkaConfig.java`, `application.properties` (puerto 5030), serializadores JSON e idempotencia.
Productores y consumidores en dos tópicos con DTOs estructurados.	12%	**100% (Muy Bueno)**	Tópicos `carhub.ventas.streaming` y `carhub.actividad.vehiculos` con DTOs tipados.
Microservicio administrador de Kafka (endpoints/clases para crear, listar, administrar).	6%	**100% (Muy Bueno)**	`KafkaAdminController` y `KafkaAdminService` en `backend-carhub` sobre `AdminClient`.
Productores Kafka correctamente implementados con DTOs y serializers.	14%	**100% (Muy Bueno)**	`VentaKafkaProducer` y `VehiculoActividadKafkaProducer` con callbacks asíncronos.
Consumidores Kafka con `@KafkaListener` y lógica separada de infraestructura.	14%	**100% (Muy Bueno)**	`VentasKafkaConsumer` y `VehiculoActividadKafkaConsumer` delegando a servicios de dominio.
Manejo de errores, logging contextual y control de reintentos.	12%	**100% (Muy Bueno)**	`DefaultErrorHandler` con `FixedBackOff(1000L, 3)` y `ErrorHandlingDeserializer`.

Indicador de Evaluación	Pond.	Nivel Alcanzado	Evidencia en el Proyecto
Parámetros de administración de tópicos (retención, cleanup, particiones, réplicas).	12%	**100% (Muy Bueno)**	3 particiones, factor 2, `retention.ms` (7 días y 3 días) y cleanup `delete`.
Consumer groups adecuados para balanceo de carga y resiliencia.	12%	**100% (Muy Bueno)**	`carhub-analytics-ventas-group` y `carhub-telemetria-vehiculos-group` con concurrencia 3.
Microservicio administrador o scripts de infraestructura para tópicos y consumer groups.	6%	**100% (Muy Bueno)**	Endpoints REST `/consumer-groups` y script utilitario `dockers/kafka-admin.sh`.
Puntaje Total Pauta	**100%**	**LOGRO DESTACADO**	**Solución integral, robusta y verificada mediante compilación multi-módulo.**